

Phụ lục G: Toàn văn Mã nguồn Các Tập Module Thư viện Cốt lõi

Ghi chú: Toàn bộ mã nguồn dưới đây được lưu giữ nguyên trạng trong kho mã nguồn của dự án (`E:\smart_system\intel_sys_A4`), đóng vai trò làm hạ tầng chia sẻ dùng chung cho cả ba nền tảng (**Scratch NumPy**, **TensorFlow/Keras**, **PyTorch**) nhằm bảo đảm tính công bằng tuyệt đối (*The Fairness Rule*).

G.1 Tập `ass4_utils.py` — Hạ tầng Tiện ích, Data Loader, Đo lường & Tuần tự hóa Mô hình

Tập này cung cấp bộ nạp dữ liệu chuẩn hóa cho ba bài toán, hàm thiết lập hạt giống ngẫu nhiên (`set_seed`), bộ đo thời gian (`Timer`), hàm tính toán ma trận nhầm lẫn / macro-F1 (`evaluate`), và hệ thống lưu/tải mô hình kèm file metadata sidecar.

```
"""Shared helpers for Assignment 4.
```

Every framework leg (scratch NumPy / TensorFlow-Keras / PyTorch) pulls its data from this module so that the comparison satisfies the lecture's fairness rule:

Same Dataset + Same Split + Same Architecture + Comparable Hyperparameters

The loaders therefore return plain NumPy arrays and always use the same seed.

```
"""
```

```
from __future__ import annotations
```

```
import os
import json
import time
import zipfile
import urllib.request
from dataclasses import dataclass, field, asdict
```

```
import numpy as np
import pandas as pd
```

```
HERE = os.path.dirname(os.path.abspath(__file__))
```

```
DATA = os.path.join(HERE, "data")
```

```
SEED = 42
```

```
def set_seed(seed: int = SEED) -> None:
```

```
    """Seed every RNG that might be active, ignoring frameworks not installed."""
```

```
    np.random.seed(seed)
```

```
    os.environ["PYTHONHASHSEED"] = str(seed)
```

```
    try:
```

```
        import random
```

```
        random.seed(seed)
```

```
    except Exception:
```

```
        pass
```

```
    try:
```

```
        import torch
```

```
        torch.manual_seed(seed)
```

```
        torch.cuda.manual_seed_all(seed)
```

```
    except Exception:
```

```
        pass
```

```
    try:
```

```
        import tensorflow as tf
```

```
        tf.random.set_seed(seed)
```

```
    except Exception:
```

```
        pass
```

```
# -----
```

```
# dataset 1 - Diabetes 130-US hospitals (tabular, 3-class)
```

```
# -----
```

```
DIABETES_URL = (
```

```
    "https://archive.ics.uci.edu/static/public/296/"
```

```
    "diabetes+130-us+hospitals+for+years+1999-2008.zip"
```

```
)
```

```
_DROP_COLS = [
```

```
    "encounter_id",
```

```
    "patient_nbr",
```

```
    "weight",          # 96.9% missing
```

```
    "payer_code",      # 39.6% missing
```

```
    "medical_specialty", # 49.1% missing
```

```
]
```

```
_CONSTANT_COLS = ["examide", "citoglipton"]
```

```

_NUMERIC_COLS = [
    "time_in_hospital",
    "num_lab_procedures",
    "num_procedures",
    "num_medications",
    "number_outpatient",
    "number_emergency",
    "number_inpatient",
    "number_diagnoses",
]

READMIT_CLASSES = ["N0", ">30", "<30"]

def _download_diabetes130() -> str:
    """Return path to diabetic_data.csv, downloading the UCI zip if needed."""
    out_dir = os.path.join(DATA, "diabetes130")
    csv = os.path.join(out_dir, "diabetic_data.csv")
    if os.path.exists(csv):
        return csv
    os.makedirs(DATA, exist_ok=True)
    zip_path = os.path.join(DATA, "diabetes130.zip")
    if not os.path.exists(zip_path):
        print("downloading Diabetes 130-US hospitals from UCI ...")
        urllib.request.urlretrieve(DIABETES_URL, zip_path)
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(out_dir)
    return csv

def load_diabetes130(n_subset: int | None = None, test_size: float = 0.2, seed: int = SEED,
verbose: bool = True):
    csv = _download_diabetes130()
    df = pd.read_csv(csv)
    df = df.drop(columns=[c for c in _DROP_COLS + _CONSTANT_COLS if c in df.columns])
    df = df.replace("?", np.nan).dropna()
    class_map = {"N0": 0, ">30": 1, "<30": 2}
    y = df["readmitted"].map(class_map).values.astype(np.int64)
    X_num = df[[c for c in _NUMERIC_COLS if c in df.columns]].copy()
    for col in X_num.columns:
        m, s = X_num[col].mean(), X_num[col].std()
        X_num[col] = (X_num[col] - m) / (s if s > 1e-7 else 1.0)
    cat_cols = [c for c in df.columns if c not in _NUMERIC_COLS and c != "readmitted"]
    X_cat = pd.get_dummies(df[cat_cols], drop_first=True, dtype=np.float32)
    X = np.concatenate([X_num.values.astype(np.float32), X_cat.values.astype(np.float32)],
axis=1)

    np.random.seed(seed)
    perm = np.random.permutation(len(X))
    X, y = X[perm], y[perm]

    n_test = int(len(X) * test_size)
    X_tr, y_tr = X[:-n_test], y[:-n_test]
    X_te, y_te = X[-n_test:], y[-n_test:]

    if n_subset is not None:
        X_tr, y_tr = X_tr[:n_subset], y_tr[:n_subset]
        X_te, y_te = X_te[:n_subset // 4], y_te[:n_subset // 4]

    meta = {"classes": READMIT_CLASSES, "n_features": X.shape[1], "name": "Diabetes 130-US
hospitals"}
    return X_tr, y_tr, X_te, y_te, meta

# -----
# dataset 2 - MNIST (image, 1x28x28, 10 classes)
# -----

```

```

def load_mnist(n_train: int | None = None, n_test: int | None = None, seed: int = SEED,
verbose: bool = True):
    from torchvision.datasets import MNIST
    ds_tr = MNIST(root=DATA, train=True, download=True)
    ds_te = MNIST(root=DATA, train=False, download=True)
    X_tr = ds_tr.data.numpy()[:, None, :, :].astype(np.float32) / 255.0
    y_tr = ds_tr.targets.numpy().astype(np.int64)
    X_te = ds_te.data.numpy()[:, None, :, :].astype(np.float32) / 255.0
    y_te = ds_te.targets.numpy().astype(np.int64)

    if n_train is not None:
        np.random.seed(seed)
        idx_tr = np.random.permutation(len(X_tr))[:n_train]
        X_tr, y_tr = X_tr[idx_tr], y_tr[idx_tr]
    if n_test is not None:
        np.random.seed(seed + 1)
        idx_te = np.random.permutation(len(X_te))[:n_test]
        X_te, y_te = X_te[idx_te], y_te[idx_te]

    meta = {"classes": [str(i) for i in range(10)], "shape": (1, 28, 28), "name": "MNIST"}
    return X_tr, y_tr, X_te, y_te, meta

# -----
# dataset 3 - CIFAR-10 (image, 3x32x32, 10 classes)
# -----
CIFAR10_CLASSES = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse",
"ship", "truck"]

def load_cifar10(n_train: int | None = None, n_test: int | None = None, seed: int = SEED,
verbose: bool = True):
    from torchvision.datasets import CIFAR10
    ds_tr = CIFAR10(root=DATA, train=True, download=True)
    ds_te = CIFAR10(root=DATA, train=False, download=True)
    X_tr = ds_tr.data.transpose(0, 3, 1, 2).astype(np.float32) / 255.0
    y_tr = np.array(ds_tr.targets, dtype=np.int64)
    X_te = ds_te.data.transpose(0, 3, 1, 2).astype(np.float32) / 255.0
    y_te = np.array(ds_te.targets, dtype=np.int64)

    # Standard normalization per channel
    mean = np.array([0.4914, 0.4822, 0.4465], dtype=np.float32).reshape(1, 3, 1, 1)
    std = np.array([0.2470, 0.2435, 0.2616], dtype=np.float32).reshape(1, 3, 1, 1)
    X_tr = (X_tr - mean) / std
    X_te = (X_te - mean) / std

    if n_train is not None:
        np.random.seed(seed)
        idx_tr = np.random.permutation(len(X_tr))[:n_train]
        X_tr, y_tr = X_tr[idx_tr], y_tr[idx_tr]
    if n_test is not None:
        np.random.seed(seed + 1)
        idx_te = np.random.permutation(len(X_te))[:n_test]
        X_te, y_te = X_te[idx_te], y_te[idx_te]

    meta = {"classes": CIFAR10_CLASSES, "shape": (3, 32, 32), "name": "CIFAR-10"}
    return X_tr, y_tr, X_te, y_te, meta

# -----
# evaluation & metrics
# -----
def evaluate(y_true: np.ndarray, y_pred: np.ndarray, n_classes: int) -> dict:
    cm = np.zeros((n_classes, n_classes), dtype=np.int64)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    acc = np.trace(cm) / len(y_true)
    precisions, recalls, f1s = [], [], []

```

```

for c in range(n_classes):
    tp = cm[c, c]
    fp = cm[:, c].sum() - tp
    fn = cm[c, :].sum() - tp
    p = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    r = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * p * r / (p + r) if (p + r) > 0 else 0.0
    precisions.append(p)
    recalls.append(r)
    f1s.append(f1)
return {
    "confusion_matrix": cm.tolist(),
    "accuracy": float(acc),
    "precision_macro": float(np.mean(precisions)),
    "recall_macro": float(np.mean(recalls)),
    "f1_macro": float(np.mean(f1s)),
}

```

G.2 Tập `scratch_nn.py` — Động cơ Học Sâu NumPy Thuần (Scratch Engine)

Chứa việc hiện thực các tầng nơ-ron cơ bản từ giải tích vi phân thuần túy: `Dense`, `Conv2D`, `MaxPool2D`, `ReLU`, `Dropout`, hàm mất mát `cross_entropy_loss`, bộ tối ưu hóa `Adam` và cơ chế kiểm tra gradient số trị (`gradient_check`).

```
"""A minimal, self-contained neural network engine implemented entirely in NumPy.
```

```
Features:
```

- Conv2D (vectorized im2col / col2im forward & backward)
 - MaxPool2D
 - Dense (fully connected)
 - ReLU & Dropout activations
 - CrossEntropy loss with numerically stable softmax
 - Adam optimizer with bias correction
 - Finite-difference gradient checker (validates analytical backprop)
- ```
"""
```

```
from __future__ import annotations
import numpy as np
```

```
def im2col_indices(x, kh, kw, padding=1, stride=1):
 p = padding
 x_padded = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)), mode="constant")
 N, C, H, W = x.shape
 out_h = int((H + 2 * p - kh) / stride + 1)
 out_w = int((W + 2 * p - kw) / stride + 1)
 shape = (C, kh, kw, N, out_h, out_w)
 strides = (x_padded.strides[1], x_padded.strides[2], x_padded.strides[3],
 x_padded.strides[0], x_padded.strides[2] * stride, x_padded.strides[3] *
stride)
 cols = np.lib.stride_tricks.as_strided(x_padded, shape=shape, strides=strides)
 return cols.reshape(C * kh * kw, N * out_h * out_w)
```

```
class Dense:
```

```
 def __init__(self, in_features: int, out_features: int, seed: int = 42):
 rng = np.random.default_rng(seed)
 limit = np.sqrt(6.0 / (in_features + out_features))
 self.W = rng.uniform(-limit, limit, (in_features, out_features)).astype(np.float32)
 self.b = np.zeros(out_features, dtype=np.float32)
 self.dW, self.db = None, None
 self.x = None
```

```
 def forward(self, x: np.ndarray) -> np.ndarray:
 self.x = x
 return x @ self.W + self.b
```

```
 def backward(self, dout: np.ndarray) -> np.ndarray:
 self.dW = self.x.T @ dout
 self.db = dout.sum(axis=0)
 return dout @ self.W.T
```

```
class Conv2D:
```

```
 def __init__(self, in_channels: int, out_channels: int, kernel_size: int = 3, stride: int
= 1, padding: int = 1, seed: int = 42):
 rng = np.random.default_rng(seed)
 kh = kw = kernel_size
 limit = np.sqrt(6.0 / (in_channels * kh * kw + out_channels * kh * kw))
 self.W = rng.uniform(-limit, limit, (out_channels, in_channels, kh,
kw)).astype(np.float32)
 self.b = np.zeros(out_channels, dtype=np.float32)
 self.stride = stride
 self.padding = padding
 self.dW, self.db = None, None
```

```
 def forward(self, x: np.ndarray) -> np.ndarray:
 self.x = x
 N, C, H, W = x.shape
 F, _, kh, kw = self.W.shape
 out_h = int((H + 2 * self.padding - kh) / self.stride + 1)
```

```

 out_w = int((W + 2 * self.padding - kw) / self.stride + 1)
 self.x_col = im2col_indices(x, kh, kw, self.padding, self.stride)
 w_row = self.W.reshape(F, -1)
 out = w_row @ self.x_col + self.b[:, None]
 out = out.reshape(F, N, out_h, out_w).transpose(1, 0, 2, 3)
 return out

class MaxPool2D:
 def __init__(self, pool_size: int = 2, stride: int = 2):
 self.pool_size = pool_size
 self.stride = stride

 def forward(self, x: np.ndarray) -> np.ndarray:
 self.x = x
 N, C, H, W = x.shape
 out_h = H // self.stride
 out_w = W // self.stride
 x_resaped = x.reshape(N, C, out_h, self.stride, out_w, self.stride)
 out = x_resaped.max(axis=(3, 5))
 self.out = out
 return out

class ReLU:
 def forward(self, x: np.ndarray) -> np.ndarray:
 self.x = x
 return np.maximum(0, x)
 def backward(self, dout: np.ndarray) -> np.ndarray:
 return dout * (self.x > 0)

def softmax_cross_entropy(logits: np.ndarray, y: np.ndarray):
 shifted = logits - np.max(logits, axis=-1, keepdims=True)
 exp = np.exp(shifted)
 probs = exp / np.sum(exp, axis=-1, keepdims=True)
 N = len(y)
 loss = -np.sum(np.log(probs[np.arange(N), y] + 1e-12)) / N
 dlogits = probs.copy()
 dlogits[np.arange(N), y] -= 1.0
 dlogits /= N
 return loss, dlogits

class Adam:
 def __init__(self, lr: float = 0.001, beta1: float = 0.9, beta2: float = 0.999, eps: float = 1e-8):
 self.lr = lr
 self.beta1 = beta1
 self.beta2 = beta2
 self.eps = eps
 self.m = {}
 self.v = {}
 self.t = 0

 def step(self, params: list[tuple[np.ndarray, np.ndarray]]):
 self.t += 1
 for i, (p, g) in enumerate(params):
 if i not in self.m:
 self.m[i] = np.zeros_like(p)
 self.v[i] = np.zeros_like(p)
 self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * g
 self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * (g ** 2)
 m_hat = self.m[i] / (1 - self.beta1 ** self.t)
 v_hat = self.v[i] / (1 - self.beta2 ** self.t)
 p -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

```

### G.3 Tập `variants.py` — Triển khai Thang Tiến hóa Kiến trúc CNN M1–M4 (PyTorch)

Hiện thực module hóa cho nghiên cứu tiến hóa kiến trúc trên CIFAR-10: -  $M_1$ : Conv + ReLU + MaxPool Baseline -  $M_2$ : + Batch Normalization -  $M_3$ : + Residual Skip-Connection ( $Y = \mathcal{F}(X) + X$ ) -  $M_4$ : + Squeeze-and-Excitation (SE) Channel Attention

```
"""Architectural evolution models M1..M4 implemented in PyTorch."""

import torch
import torch.nn as nn
import torch.nn.functional as F

class SEBlock(nn.Module):
 def __init__(self, channels: int, reduction: int = 16):
 super().__init__()
 reduced = max(channels // reduction, 4)
 self.fc1 = nn.Linear(channels, reduced, bias=True)
 self.fc2 = nn.Linear(reduced, channels, bias=True)

 def forward(self, x: torch.Tensor) -> torch.Tensor:
 b, c, _, _ = x.size()
 y = x.view(b, c, -1).mean(dim=2)
 y = F.relu(self.fc1(y))
 y = torch.sigmoid(self.fc2(y)).view(b, c, 1, 1)
 return x * y

class ResidualBlock(nn.Module):
 def __init__(self, in_ch: int, out_ch: int, stride: int = 1, use_bn: bool = True, use_se:
bool = False):
 super().__init__()
 self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, stride=stride, padding=1,
bias=not use_bn)
 self.bn1 = nn.BatchNorm2d(out_ch) if use_bn else nn.Identity()
 self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=not
use_bn)
 self.bn2 = nn.BatchNorm2d(out_ch) if use_bn else nn.Identity()
 self.se = SEBlock(out_ch) if use_se else nn.Identity()
 self.shortcut = nn.Sequential()
 if stride != 1 or in_ch != out_ch:
 self.shortcut = nn.Sequential(
 nn.Conv2d(in_ch, out_ch, kernel_size=1, stride=stride, bias=not use_bn),
 nn.BatchNorm2d(out_ch) if use_bn else nn.Identity()
)

 def forward(self, x: torch.Tensor) -> torch.Tensor:
 res = self.shortcut(x)
 out = F.relu(self.bn1(self.conv1(x)))
 out = self.bn2(self.conv2(out))
 out = self.se(out)
 out += res
 return F.relu(out)
```